Para correr el código es necesario hacer los siguientes pasos:

PowerShell
usbipd list # Busca el BUSID de tu dispositivo
usbipd bind --busid <BUSID>
usbipd attach --wsl --busid <BUSID>

Terminal wsl
source ~/.profile && source ~/.bashrc && get_esp32 && idf.py
set-target esp32s3 && idf.py build && idf.py -p /dev/ttyUSB0
flash monitor

Para correr el codigo despues de cambiar la frecuencia
idf.py fullclean && idf.py build && idf.py -p /dev/ttyUSB0 flash monitor

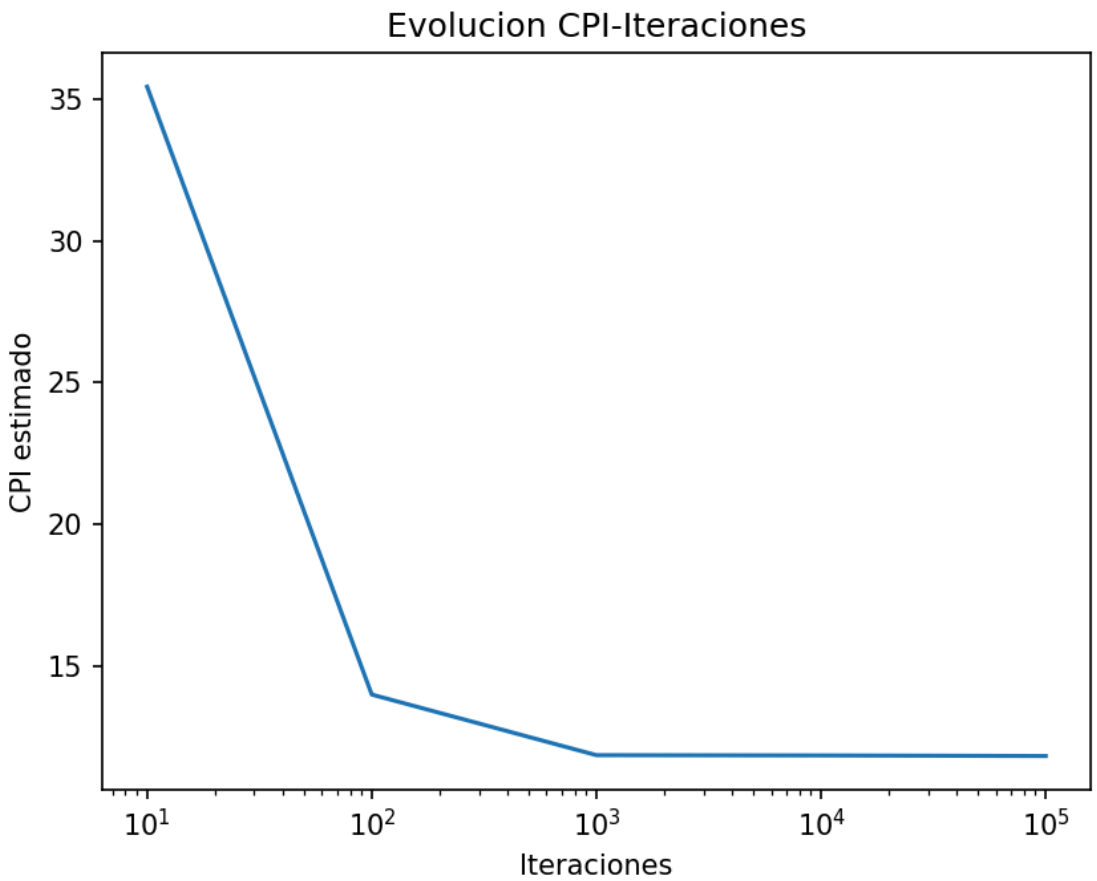
Ejercicio 3:

3.1: Determine cómo evoluciona la medida CPI cuando incrementa el valor de X, analice el resultado. Entregable: Gráfico + Análisis

Iteraciones (X)	**CPI Estimado**
10	35.44
100	13.98
1,000	11.84
10,000	11.83
100,000	11.81

```
diazb@DESKTOP-HGLIQB2: ~/Proyecto-Sumo-grupo-Juan-Lucas-Cristobal/Lab_1/Ejercicio_3
I (176) app_init: ELF file SHA256:  a795ca49f...
I (180) app_init: ESP-IDF:          v6.1-dev-2938-g12f36a021f
I (186) efuse_init: Min chip rev:     v0.0
I (189) efuse_init: Max chip rev:     v0.99
I (193) efuse_init: Chip rev:         v0.2
I (197) heap_init: Initializing. RAM available for dynamic allocation:
I (204) heap_init: At 3FC93A40 len 00055CD0 (343 KiB): RAM
I (209) heap_init: At 3FCE9710 len 00005724 (21 KiB): RAM
I (214) heap_init: At 3FCF0000 len 00008000 (32 KiB): DRAM
I (219) heap_init: At 600FE000 len 00001FE8 (7 KiB): RTCRAM
I (225) spi_flash: detected chip: mxic
I (228) spi_flash: flash io: dio
W (231) spi_flash: Detected size(16384k) larger than the size in the binary image header(2048k). Using the size in the b
inary image header.
I (243) sleep_gpio: Configure to isolate all GPIO pins in sleep state
I (250) sleep_gpio: Enable automatic switching of GPIO sleep configuration
I (257) main_task: Started on CPU0
I (277) main_task: Calling app_main()

--- RESULTADOS EJERCICIO 3 ---
Iteraciones (X): 1000
Frecuencia CPU: 160 MHz
Tiempo medido (esp_timer): 370 us
Tiempo calculado (ciclos): 0.000369 s
Ciclos totales: 59024
CPI estimado: 11.8048
Check Results: PASS
-----------------------------
I (287) main_task: Returned from app_main()
```



Se observa que para valores bajos de X (X=10), el CPI es significativamente mayor (35.44). Esto se explica por el overhead del sistema: las llamadas a funciones de medición como esp_cpu_get_cycle_count() y la preparación del bucle for consumen ciclos que, al ser pocos los datos procesados, inflan el promedio por instrucción.

A medida que X aumenta, este costo fijo se diluye entre un número mayor de instrucciones. A partir de X=1,000, el CPI se estabiliza en un valor aproximado de 11.81. Este valor representa el costo real de ejecución de las operaciones aritméticas (suma, multiplicación, división y módulo) en la arquitectura del ESP32-S3 bajo las restricciones impuestas por el uso de variables volatile

Efecto del Overhead Inicial: Observamos que para X=10, el CPI es desproporcionadamente alto (35.44). Esto se debe a que el costo de inicializar el cronómetro (esp_timer_get_time), configurar el bucle y las primeras instrucciones de salto (overhead) se divide entre muy pocas operaciones. A medida que X aumenta, este costo inicial se vuelve insignificante frente al total.

Estabilización del CPI: Entre X=1,000 y X=100,000, el CPI se estabiliza cerca de 11.81. Esto indica que el procesador ha alcanzado su "estado estacionario" de ejecución para este bloque de código específico.

Influencia de la arquitectura: Un CPI cercano a 12 es alto para operaciones aritméticas. Esto sugiere que el uso de variables volatile (que obligan a acceder a memoria RAM en cada paso) y la presencia de operaciones complejas como la división (/) y el módulo (%) están penalizando el rendimiento por ciclo.

3.2:

Prueba A:
La línea result_2 = var_1 % var_2; se comentó
--- RESULTADOS EJERCICIO 3 ---
Iteraciones (X): 1000
Tiempo total: 288 us
Ciclos totales: 45858
CPI estimado: 9.17
Check Results: PASS

Prueba B:
La línea result_4 = var_1 / var_2; se comentó
--- RESULTADOS EJERCICIO 3 ---
Iteraciones (X): 1000
Tiempo total: 294 us
Ciclos totales: 46856
CPI estimado: 9.37
Check Results: PASS

Prueba C: Se comentó la multiplicación

--- RESULTADOS EJERCICIO 3 ---
Iteraciones (X): 1000

Tiempo total: 300 us
Ciclos totales: 47856
CPI estimado: 9.57
Check Results: PASS

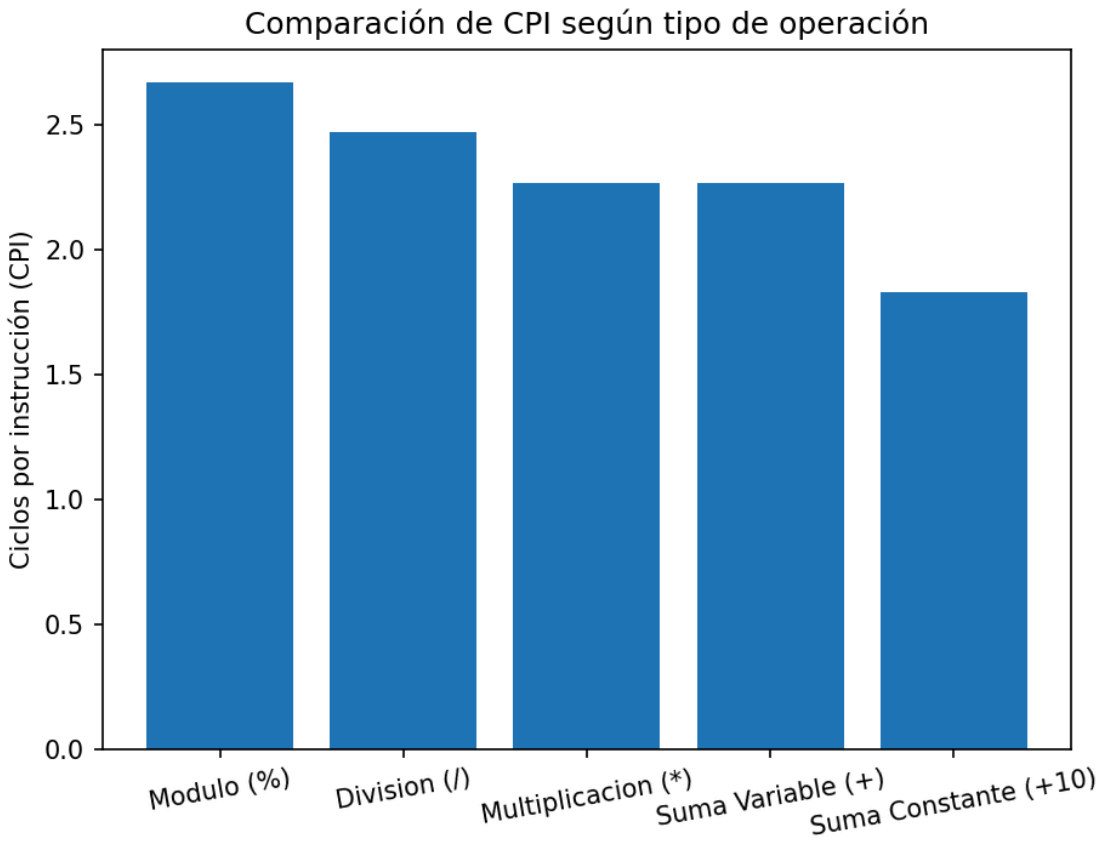
Prueba D: Se comentó la suma de variables
--- RESULTADOS EJERCICIO 3 ---
Iteraciones (X): 1000
Tiempo total: 300 us
Ciclos totales: 47854
CPI estimado: 9.571
Check Results: PASS

Prueba E: Se comentó la suma de constante (+10)
--- RESULTADOS EJERCICIO 3 ---
Iteraciones (X): 1000
Tiempo total: 314 us
Ciclos totales: 50041
CPI estimado: 10.0082
Check Results: PASS

Para identificar la operación más costosa, se realizaron pruebas de descarte comentando una instrucción a la vez y comparando el resultado con el CPI base de **11.84**

Operación Comentada	**Ciclos Totales**	**Reducción (Ciclos aprox.)**	**Impacto en el CPI**	**Reducción de CPI**
Ninguna (Base)	59,192	0	11.84	0.00
Módulo (%) (Prueba A)	45,858	13,334	9.17	2.67
División (/) (Prueba B)	46,856	12,336	9.37	2.47
Multiplicación (*) (Prueba D)	47,856	11,336	9.57	2.27
Suma Variable (+) (Prueba E)	47,854	11,338	9.57	2.27

Suma Constante (+10) (Prueba F)	50,041	9,151	10.01	1.83



Identificación del Bottleneck

Tras analizar la reducción del CPI, se determina que el cuello de botella (bottleneck) del programa es la operación de Módulo (%), seguida estrechamente por la División (/).

Impacto cuantitativo: Al eliminar la operación de módulo, el CPI del sistema cae en 2.67 puntos, la mayor mejora de eficiencia observada en todas las pruebas.

Complejidad Arquitectónica: En la arquitectura Xtensa del ESP32-S3, mientras que las sumas y multiplicaciones suelen ejecutarse en pocos ciclos gracias a unidades dedicadas, las operaciones de división y módulo son más complejas. Estas requieren múltiples ciclos de reloj para completar el algoritmo de división o, en su defecto, llamadas a rutinas de software que penalizan significativamente el rendimiento.

Análisis del impacto de volatile y Memoria

El hecho de que las operaciones de Suma y Multiplicación muestren reducciones de CPI similares (2.27) sugiere que gran parte del costo de estas instrucciones "simples" no proviene de la unidad aritmética, sino del uso de variables de tipo volatile.

Esta directiva obliga al procesador a realizar un acceso a la memoria RAM en cada iteración, impidiendo optimizaciones de registro que el compilador aplicaría normalmente.

Sin embargo, incluso con este costo de acceso a memoria presente en todas las pruebas, el Módulo y la División sobresalen como las operaciones más lentas debido a su costo computacional intrínseco.

Conclusión

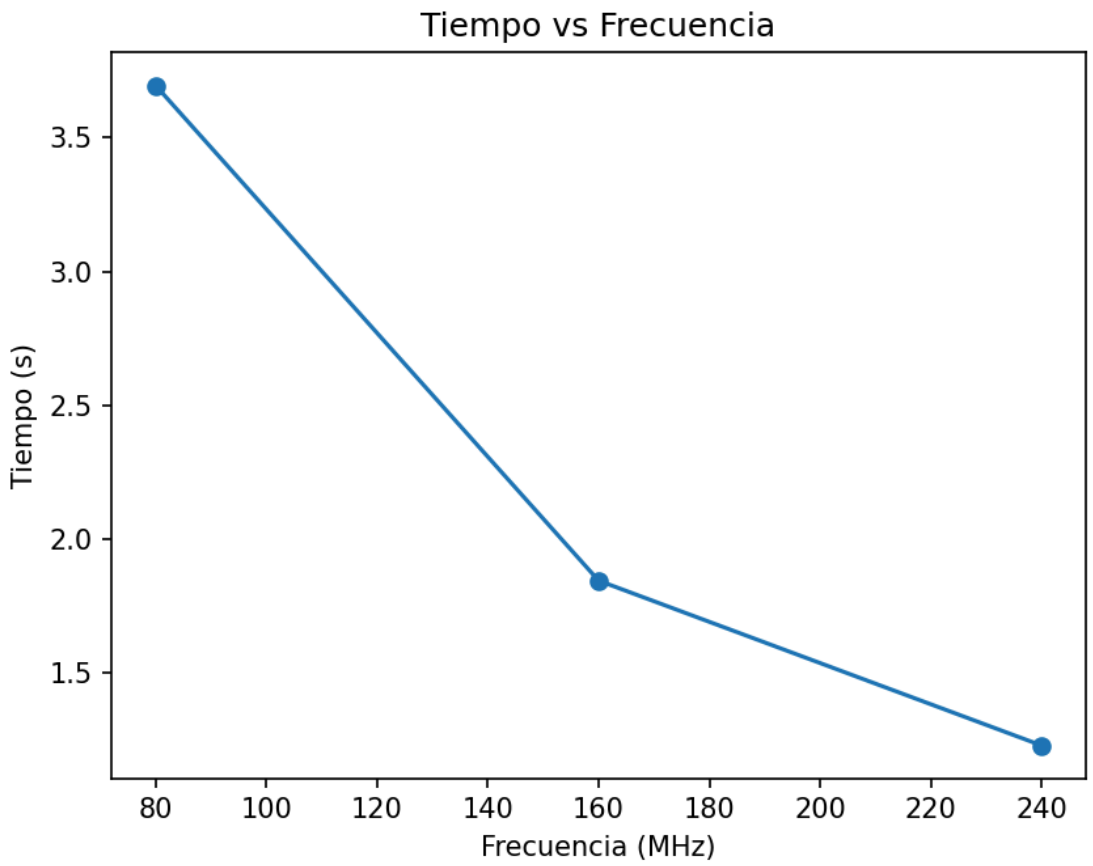
Para optimizar el desempeño de este programa, la prioridad debe ser minimizar el uso de divisiones y módulos dentro de bucles críticos, o intentar sustituirlos por operaciones de desplazamiento de bits (bit-shifting) cuando sea posible, dado que estas últimas operan con un CPI mucho menor.

3.3

Frecuencia (MHz)	Tiempo (µs)	Ciclos Totales	Potencia Pico (W)	Energía (J)

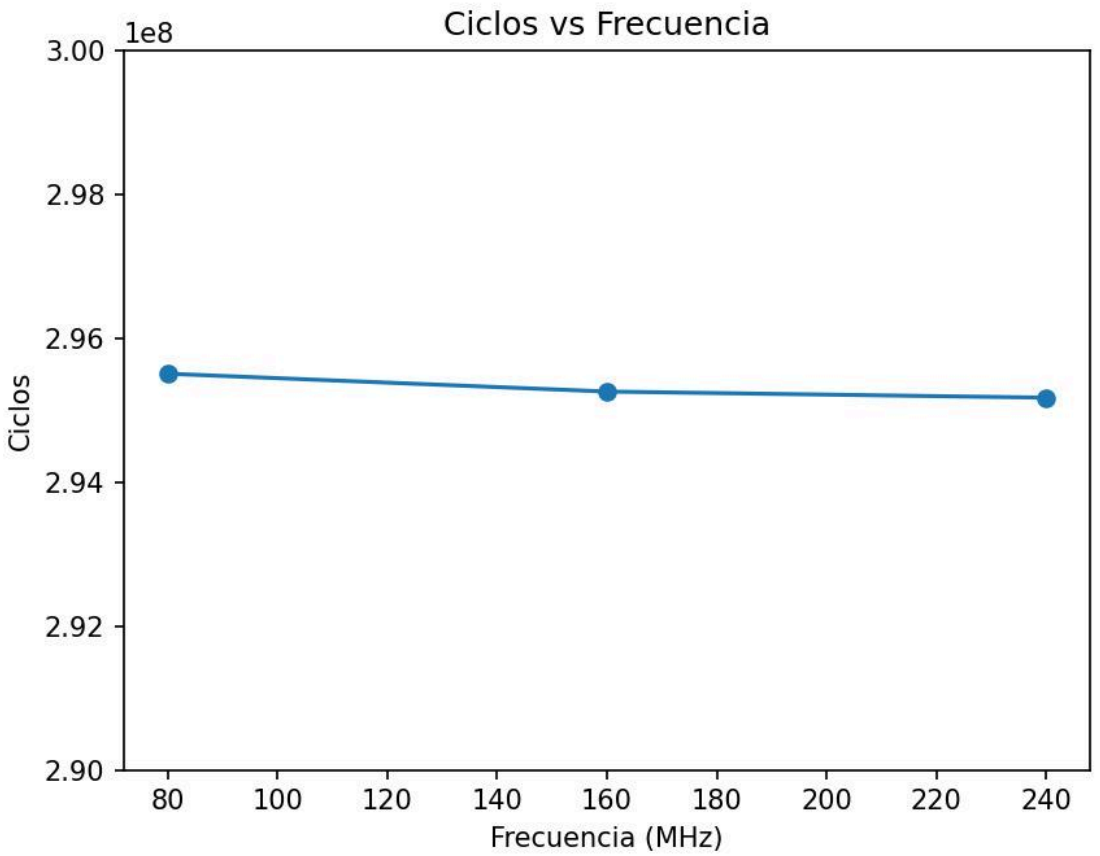
80 MHz	3.693.913	295.512.941	0.3741 W	1.382 J
160 MHz	1.845.408	295.264.974	0.4201 W	0.775 J
240 MHz	1.229.918	295.180.045	0.4739 W	0.582 J

1. Tiempo vs. Frecuencia



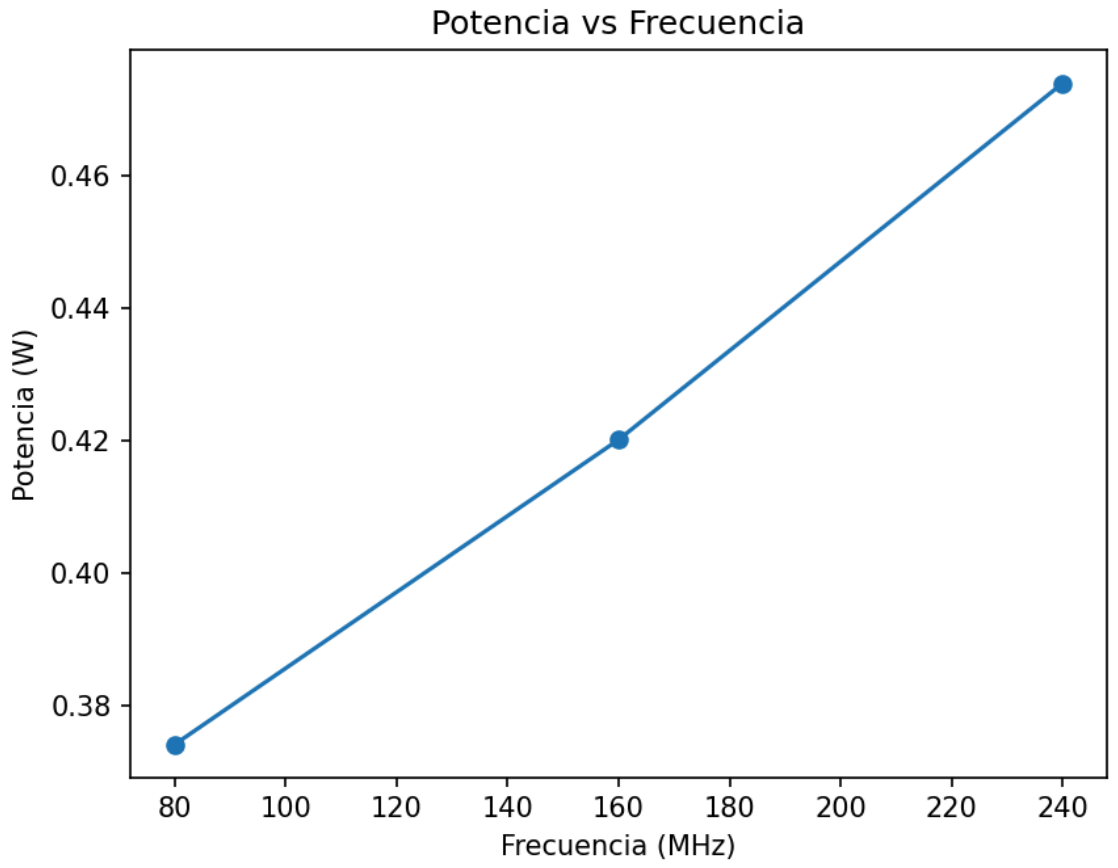
El tiempo de ejecución es inversamente proporcional a la frecuencia. A 80 MHz, el programa tarda casi 3 veces más que a 240 MHz. Esto demuestra que para tareas intensivas en cómputo, bajar la frecuencia penaliza fuertemente el tiempo de respuesta.

2. Ciclos de Reloj y CPI



Se puede ver como los Ciclos Totales se mantienen prácticamente constantes (295 millones) en las tres pruebas. Esto se debe a que la carga de trabajo (el bucle for y sus operaciones) no cambia, solo cambia la velocidad a la que el procesador los ejecuta. El CPI se mantiene estable en 11.8, lo que indica que la eficiencia por ciclo de la arquitectura es independiente de la frecuencia del reloj.

3. Potencia y Energía



El gráfico de Potencia vs Frecuencia confirma una correlación lineal positiva. A 80 MHz, el consumo es de 0.37 W, incrementándose un 26% al alcanzar los 240 MHz (0.47 W). Este fenómeno es consistente con la teoría de circuitos CMOS, donde la potencia dinámica aumenta proporcionalmente a la frecuencia de conmutación .

Aunque la operación a 240 MHz demanda la mayor potencia instantánea, resulta ser la más eficiente en términos de Energía Total (0.582 J). Esto ocurre porque la reducción en el tiempo de ejecución (de 3.69s a 1.23s) compensa el aumento en potencia.

En este escenario de alta carga computacional (CPU-bound), la estrategia Race to Sleep (ejecutar a máxima velocidad para entrar en reposo lo antes posible) es superior para el ahorro de batería. Ir más lento (80 MHz) obliga al sistema a mantener encendidos sus componentes base y periféricos durante casi el triple del tiempo, pagando un "costo fijo" energético (overhead) innecesario que eleva el consumo total a 1.382 J.

--- RESULTADOS EJERCICIO 3 ---
Iteraciones (X): 5000000
Frecuencia CPU: 80 MHz
Tiempo medido (esp_timer): 3693913 us
Tiempo calculado (ciclos): 3.693912 s

```
Ciclos totales: 295512941
CPI estimado: 11.8205
Check Results: PASS

------------------------------
--- RESULTADOS EJERCICIO 3 ---
Iteraciones (X): 5000000
Frecuencia CPU: 160 MHz
Tiempo medido (esp_timer): 1845408 us
Tiempo calculado (ciclos): 1.845406 s
Ciclos totales: 295264974
CPI estimado: 11.8106
Check Results: PASS
------------------------------

--- RESULTADOS EJERCICIO 3 ---
Iteraciones (X): 5000000
Frecuencia CPU: 240 MHz
Tiempo medido (esp_timer): 1229918 us
Tiempo calculado (ciclos): 1.229917 s
Ciclos totales: 295180045
CPI estimado: 11.8072
Check Results: PASS
------------------------------
```